PPS Technical Deep Dive - Part 2: Architecture & Technology

ErthaLoka Planetary Performance System *Di-MRVM (Digital Measure, Report, Verify, Monitor)*

Table of Contents

1. System Architecture Overview
2. Technology Stack
3. Frontend Architecture
4. Backend Architecture
5. Google Earth Engine Integration
6. External API Integration
7. Database Architecture
8. API Specifications
9. Data Sources & Satellite Constellations
10. Processing Pipeline
11. Security & Authentication
12. Deployment Architecture

---

1. System Architecture Overview

High-Level Architecture Diagram

```
+-----------------------------------------------------------------+
|                        CLIENT LAYER                              |
|  +-------------------------------------------------------------+  |
|  |  React + TypeScript Frontend (Vite)                          |  |
|  |  - Interactive Map (Leaflet + React-Leaflet)                 |  |
|  |  - Charts (Recharts + Chart.js)                              |  |
|  |  - UI Components (Radix UI + shadcn/ui)                      |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------------------------------------+
                              |
                    REST API (JSON/HTTP)
                              |
+-----------------------------------------------------------------+
|                      API GATEWAY LAYER                           |
|  +-------------------------------------------------------------+  |
|  |  FastAPI Server (Python)                                     |  |
|  |  - Query Orchestration                                       |  |
|  |  - Request Validation (Pydantic)                             |  |
|  |  - CORS Middleware                                           |  |
|  |  - Rate Limiting                                             |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------------------------------------+
                              |
        +-----------+---------+---------+-----------+
        |           |         |         |           |
+-------v---+ +-----v-----+ +-v-------+ +-v-------+ +--v--------+
| Pillar A  | | Pillar B  | | Pillar C| | Pillar D| | Pillar E  |
| Atmosph.  | | Biodiver. | | Carbon  | | Degrad. | | Ecosystem |
+-----------+ +-----------+ +---------+ +---------+ +-----------+
        |           |         |         |           |
        +-----------+---------+---------+-----------+
```

```
                              |
+-----------------------------------------------------------------+
|                   DATA PROCESSING LAYER                          |
|  +-------------------------------------------------------------+  |
|  |  - Normalization Engine (6 algorithms)                      |  |
|  |  - Scoring Engine (Weighted aggregation)                    |  |
|  |  - Quality Assessment (DQS calculation)                     |  |
|  |  - ESV Calculator (Costanza framework)                      |  |
|  |  - Carbon Credit Calculator (IPCC Tier 1)                   |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------------------------------------+
                              |
+-----------------------------------------------------------------+
|                    DATA SOURCE LAYER                             |
|  +----------------+  +----------------+  +--------------------+   |
|  | Google Earth   |  | External APIs  |  | IoT Sensors        |   |
|  | Engine (GEE)   |  | - Open-Meteo   |  | - Firebase/        |   |
|  | - MODIS        |  | - OpenAQ       |  |   Firestore        |   |
|  | - Sentinel-2   |  | - Weather      |  | - Real-time        |   |
|  | - Landsat      |  |                |  |   monitoring       |   |
|  | - GEDI/SRTM    |  |                |  |                    |   |
|  +----------------+  +----------------+  +--------------------+   |
+-----------------------------------------------------------------+
                              |
+-----------------------------------------------------------------+
|                    STORAGE LAYER                                 |
|  +--------------------+  +--------------------+                  |
|  | Supabase (PostgreSQL)| | Firebase Firestore |                |
|  | - Query history      | | - Sensor data      |                |
|  | - User profiles      | | - Real-time streams|                |
|  | - Organizations      | | - Device registry  |                |
|  +--------------------+  +--------------------+                  |
+-----------------------------------------------------------------+
```

Architecture Principles

1. **Microservice-Oriented**: Each pillar operates as independent query module
2. **Parallel Processing**: 5 pillars queried concurrently using ThreadPoolExecutor
3. **Caching Strategy**: In-memory caching for repeated queries
4. **Graceful Degradation**: External API fallback when satellite data unavailable
5. **API-First Design**: RESTful endpoints with OpenAPI documentation

---

2. Technology Stack

Frontend Stack

Category	Technology	Version	Purpose
Core Framework	React	^18.3.1	UI library
Language	TypeScript	^5.8.3	Type-safe development
Build Tool	Vite	^5.4.19	Fast bundling & HMR
Styling	TailwindCSS	^3.4.17	Utility-first CSS
State Management	TanStack Query	^5.83.0	Server state management

Routing	React Router DOM	^6.30.1	Client-side routing
Form Handling	React Hook Form	^7.61.1	Form validation
Schema Validation	Zod	^3.25.76	Runtime type checking

UI Components

Category	Technology	Purpose
Component Library	Radix UI	Accessible primitives
Design System	shadcn/ui	Customizable components
Icons	Lucide React	^0.462.0
Animations	Framer Motion	^12.23.24
3D Globe	COBE	^0.6.5
Particles	tsparticles	^3.9.1

Mapping & Visualization

Category	Technology	Purpose
Maps	Leaflet	^1.9.4
React Wrapper	React-Leaflet	^4.2.1
Geospatial	@turf/turf	^7.3.1
Charts	Recharts	^2.15.4
Charts (Alt)	Chart.js	^4.5.1
PDF Generation	jspdf	^3.0.4
Canvas Capture	html2canvas	^1.4.1

Backend Stack

Category	Technology	Version	Purpose
Web Framework	FastAPI	^0.104.0	Async REST API
ASGI Server	Uvicorn	^0.24.0	Production server
Data Validation	Pydantic	^2.5.0	Request/Response models
Earth Engine	earthengine-api	^0.1.380	Satellite data access
Data Processing	Pandas	^2.0.0	DataFrame operations
Numerical	NumPy	^1.24.0	Array computations
Geospatial	GeoPandas	^0.10.0	Spatial data handling
Geometry	Shapely	^1.8.0	Geometric operations

External Integrations

Category	Technology	Purpose
Database	Supabase	^2.0.0
Real-time DB	Firebase Admin	^6.2.0
Auth	Firebase	^10.14.1
Weather API	Open-Meteo	Weather & soil data
Air Quality	OpenAQ	Real-time AQI data
HTTP Client	aiohttp	^3.9.0

Development Tools

Category	Technology	Purpose
Linting	ESLint	^9.32.0
Type Checking	TypeScript-ESLint	^8.38.0
CSS Processing	PostCSS	^8.5.6
Auto-prefixer	Autoprefixer	^10.4.21
Testing	pytest	^7.4.0
Async Testing	pytest-asyncio	^0.21.0

---

3. Frontend Architecture

Project Structure

```
src/
├── components/
│   ├── ui/                 # Radix + shadcn components
│   │   ├── button.tsx
│   │   ├── card.tsx
│   │   ├── dialog.tsx
│   │   └── ...
│   ├── charts/             # Chart components
│   │   ├── ChartConfig.ts
│   │   ├── PillarRadar.tsx
│   │   └── ESVChart.tsx
│   ├── ESVBreakdown.tsx     # ESV visualization
│   └── MapComponent.tsx     # Leaflet integration
├── services/
│   ├── phiApi.ts           # PPS API client
│   ├── metricSupplementation.ts  # External data integration
│   ├── sensorApi.ts        # IoT sensor interface
│   └── apiMiddleware.ts    # Response transformation
├── types/
│   ├── phi.ts              # PPS type definitions
│   ├── weather.ts          # Weather data types
│   └── institute.ts        # Organization types
├── utils/
│   ├── esvCalculation.ts   # ESV computation
```

```
│   ├── esvProjection.ts    # 5-year projections
│   └── impactScore/        # Impact calculations
├── hooks/
│   ├── useSensorData.ts    # Real-time sensor hook
│   └── useEnvironmentalScore.ts
├── auth/
│   ├── firebase.ts         # Firebase config
│   └── authService.ts      # Auth operations
├── db/
│   ├── firestore.ts        # Firestore client
│   └── sensorService.ts    # Sensor data service
└── pages/
    ├── Dashboard.tsx
    ├── GetTheReport.tsx
    └── LandAnalysis.tsx
```

State Management Pattern

```typescript
// TanStack Query for server state
import { useQuery, useMutation } from '@tanstack/react-query';

// PHI Query Hook Example
export const usePHIQuery = (request: PHIQueryRequest) => {
  return useQuery({
    queryKey: ['phi', request.latitude, request.longitude, request.mode],
    queryFn: () => fetchPHIReport(request),
    staleTime: 5 * 60 * 1000,  // 5 minute cache
    retry: 2,
    enabled: Boolean(request.latitude && request.longitude)
  });
};
```

Component Architecture

```typescript
// Type-safe component with Radix primitives
interface PillarCardProps {
  pillar: PillarData;
  config: PillarConfig;
  onExpand?: () => void;
}

const PillarCard: React.FC<PillarCardProps> = ({ pillar, config, onExpand }) => {
  const grade = getGradeFromScore(pillar.score);
  const gradeColor = getInterpretationColor(grade);

  return (
    <Card className="relative overflow-hidden">
      <CardHeader style={{ borderTopColor: config.color }}>
        <CardTitle>{config.fullName}</CardTitle>
      </CardHeader>
      <CardContent>
        <ScoreBadge score={pillar.score} grade={grade} color={gradeColor} />
        <MetricsList metrics={pillar.metrics} />
      </CardContent>
```

```
    </Card>
  );
};
```

---

4. Backend Architecture

Project Structure

```
Hello-/
├── app.py                      # FastAPI main application
├── planetary_health_query/
│   ├── __init__.py
│   ├── core/
│   │   ├── config.py          # METRIC_PARAMS, PILLAR_CONFIG
│   │   ├── engine.py          # GEEQueryEngine orchestrator
│   │   └── authenticator.py   # GEE authentication
│   ├── pillars/
│   │   ├── __init__.py
│   │   ├── base.py            # BasePillar abstract class
│   │   ├── atmospheric.py     # Pillar A implementation
│   │   ├── biodiversity.py    # Pillar B implementation
│   │   ├── carbon.py          # Pillar C implementation
│   │   ├── degradation.py     # Pillar D implementation
│   │   └── ecosystem.py       # Pillar E implementation
│   └── utils/
│       ├── normalization.py   # 6 normalization functions
│       ├── scoring.py         # Score calculation
│       └── quality.py         # DQS calculation
└── backend/
    ├── requirements.txt
    └── external_apis/
        ├── open_meteo.py      # Weather integration
        └── openaq.py          # Air quality integration
```

GEE Query Engine

```python
# core/engine.py - Main orchestrator
class GEEQueryEngine:
    """
    Singleton query engine managing all pillar queries.
    Supports parallel execution with ThreadPoolExecutor.
    """

    def __init__(self, project_id: str = None, auto_init: bool = True):
        self.project_id = project_id or get_project_id()
        self._initialized = False

        # Initialize pillar handlers
        self._pillars = {
            "A": AtmosphericPillar(),
            "B": BiodiversityPillar(),
            "C": CarbonPillar(),
            "D": DegradationPillar(),
            "E": EcosystemPillar()
```

```python
        }

        if auto_init:
            self.initialize()

    def query(
        self,
        lat: float,
        lon: float,
        mode: str = "comprehensive",
        parallel: bool = True,
        **kwargs
    ) -> Dict[str, Any]:
        """Execute parallel pillar queries and aggregate results."""

        if parallel:
            results = self._query_parallel(lat, lon, mode, ...)
        else:
            results = self._query_sequential(lat, lon, mode, ...)

        # Add scoring, DQS, and summary
        result = self._add_scores(result)
        result["summary"] = self._create_summary(result)

        return result

    def _query_parallel(self, ...):
        """Concurrent pillar queries using ThreadPoolExecutor."""
        with ThreadPoolExecutor(max_workers=5) as executor:
            futures = {
                executor.submit(
                    self._pillars[pid].query, lat, lon, mode, ...
                ): pid
                for pid in pillar_ids
            }

            for future in as_completed(futures):
                pillar_id = futures[future]
                try:
                    results[pillar_id] = future.result()
                except Exception as e:
                    results[pillar_id] = {"error": str(e)}

        return results
```

Base Pillar Pattern

```python
# pillars/base.py - Abstract base class
class BasePillar(ABC):
    """Abstract base class for all pillar implementations."""

    PILLAR_ID: str = ""
    PILLAR_NAME: str = ""
    PILLAR_COLOR: str = "#000000"
```

```python
    @abstractmethod
    def get_simple_metrics(self) -> List[str]:
        """Return 2-3 key metrics for simple mode."""
        pass

    @abstractmethod
    def get_comprehensive_metrics(self) -> List[str]:
        """Return 4-5 metrics for comprehensive mode."""
        pass

    @abstractmethod
    def query_metrics(
        self,
        point: ee.Geometry.Point,
        buffer_radius: int,
        date_range: Tuple[str, str],
        metrics: List[str]
    ) -> Dict[str, Any]:
        """Query specific metrics from Earth Engine."""
        pass

    def query(self, lat: float, lon: float, mode: str = "comprehensive", ...) -> Dict:
        """Main query method with validation and metadata."""
        point = ee.Geometry.Point([lon, lat])
        metrics = self.get_comprehensive_metrics() if mode == "comprehensive" else self.get_simple_metrics()

        result = self.query_metrics(point, buffer_radius, date_range, metrics)
        result["pillar_id"] = self.PILLAR_ID
        result["pillar_name"] = self.PILLAR_NAME

        return result

    def _reduce_region(self, image: ee.Image, region: ee.Geometry, scale: int):
        """Reduce image over region with mean reducer."""
        return image.reduceRegion(
            reducer=ee.Reducer.mean(),
            geometry=region,
            scale=scale,
            maxPixels=1e9
        ).getInfo()
```

---

5. Google Earth Engine Integration

Authentication Flow

```python
# core/authenticator.py
import ee

def initialize_ee(project_id: str = None) -> bool:
    """
    Initialize Google Earth Engine with service account credentials.

    Authentication methods (in order of preference):
    1. Service account JSON file (GOOGLE_APPLICATION_CREDENTIALS)
```

```
    2. Cloud project default credentials
    3. Interactive OAuth (development only)
    """
    try:
        credentials = ee.ServiceAccountCredentials(
            email=SERVICE_ACCOUNT_EMAIL,
            key_file=SERVICE_ACCOUNT_KEY_PATH
        )
        ee.Initialize(credentials, project=project_id)
        return True
    except Exception as e:
        # Fallback to default credentials
        ee.Initialize(project=project_id)
        return True
```

Dataset Catalog

Dataset	GEE Collection ID	Resolution	Temporal	Metrics
MODIS NDVI	`MODIS/006/MOD13Q1`	250m	16-day	NDVI, EVI
MODIS LAI	`MODIS/061/MOD15A2H`	500m	8-day	LAI, FPAR
MODIS LST	`MODIS/061/MOD11A1`	1km	Daily	LST Day/Night
MODIS AOD	`MODIS/061/MCD19A2_GRANULES`	1km	Daily	AOD
Sentinel-2	`COPERNICUS/S2_SR_HARMONIZED`	10m	5-day	RGB, NIR
Hansen GFC	`UMD/hansen/global_forest_change_2023_v1_11`	30m	Annual	Tree Cover, Loss
WorldCover	`ESA/WorldCover/v200`	10m	Annual	Land Cover
GEDI Biomass	`LARSE/GEDI/GEDI04_A_002_MONTHLY`	1km	Monthly	Biomass
SRTM	`USGS/SRTMGL1_003`	30m	Static	Elevation
Global Human Mod.	`CSP/HM/GlobalHumanModification`	1km	Static	HMI
WorldPop	`WorldPop/GP/100m/pop`	100m	Annual	Population
JRC Water	`JRC/GSW1_4/GlobalSurfaceWater`	30m	Monthly	Water Occurrence

Earth Engine Query Pattern

```
# Example: NDVI Query
class BiodiversityPillar(BasePillar):

    def _query_ndvi(self, region: ee.Geometry, date_range: Tuple[str, str]) -> Dict:
        """Query MODIS NDVI for region."""

        collection = ee.ImageCollection('MODIS/006/MOD13Q1') \
            .filterDate(date_range[0], date_range[1]) \
            .filterBounds(region) \
            .select('NDVI')

        # Get median composite
        composite = collection.median()
```

```
        # Apply scale factor (MODIS NDVI stored as int * 10000)
        ndvi = composite.multiply(0.0001)

        # Reduce over region
        stats = ndvi.reduceRegion(
            reducer=ee.Reducer.mean(),
            geometry=region,
            scale=250,
            maxPixels=1e9
        ).getInfo()

        return {
            "value": stats.get('NDVI'),
            "unit": "index",
            "description": "Normalized Difference Vegetation Index",
            "quality": self._assess_quality(stats.get('NDVI'), 'ndvi'),
            "source": "MODIS MOD13Q1",
            "resolution": "250m"
        }
```

---

6. External API Integration

Open-Meteo Weather API

```
# backend/external_apis/open_meteo.py
import aiohttp
import openmeteo_requests
import requests_cache
from retry_requests import retry

# Setup cached session
cache_session = requests_cache.CachedSession('.cache', expire_after=3600)
retry_session = retry(cache_session, retries=5, backoff_factor=0.2)
openmeteo = openmeteo_requests.Client(session=retry_session)

async def get_weather_data(lat: float, lon: float, forecast_days: int = 7):
    """
    Fetch comprehensive weather data from Open-Meteo.

    Endpoints used:
    - /v1/forecast (hourly + daily forecasts)
    - /v1/air-quality (AQI, pollutants)
    """
    url = "https://api.open-meteo.com/v1/forecast"
    params = {
        "latitude": lat,
        "longitude": lon,
        "current": [
            "temperature_2m", "relative_humidity_2m", "precipitation",
            "weather_code", "wind_speed_10m", "wind_direction_10m"
        ],
        "hourly": [
            "temperature_2m", "precipitation_probability",
```

```
            "soil_moisture_0_to_1cm", "soil_moisture_1_to_3cm",
            "soil_moisture_3_to_9cm", "soil_moisture_9_to_27cm"
        ],
        "daily": [
            "temperature_2m_max", "temperature_2m_min",
            "precipitation_sum", "uv_index_max"
        ],
        "forecast_days": forecast_days
    }

    async with aiohttp.ClientSession() as session:
        async with session.get(url, params=params) as response:
            return await response.json()
```

OpenAQ Air Quality API

```
# backend/external_apis/openaq.py
async def get_air_quality(lat: float, lon: float, radius_km: int = 25):
    """
    Fetch real-time air quality from nearest monitoring stations.

    Parameters measured:
    - PM2.5, PM10 (particulate matter)
    - NO2 (nitrogen dioxide)
    - O3 (ozone)
    - CO (carbon monoxide)
    - SO2 (sulfur dioxide)
    """
    url = "https://api.openaq.org/v2/latest"
    params = {
        "coordinates": f"{lat},{lon}",
        "radius": radius_km * 1000,  # meters
        "limit": 10,
        "order_by": "distance"
    }

    async with aiohttp.ClientSession() as session:
        async with session.get(url, params=params, headers={"X-API-Key": API_KEY}) as response:
            data = await response.json()
            return process_openaq_response(data)
```

Metric Supplementation Strategy

```
// services/metricSupplementation.ts
const SUPPLEMENTATION_MAP = {
  // When satellite AQI is N/A, use OpenAQ
  aqi: {
    pillar: 'A',
    getValueFromExternal: (ext) => ext.air_quality?.primary_aqi ?? null,
    unit: 'US AQI',
    description: 'Real-time Air Quality Index'
  },

  // When satellite UV is N/A, use weather forecast
```

```
  uv_index: {
    pillar: 'A',
    getValueFromExternal: (ext) => ext.weather?.daily?.uv_index_max?.[0] ?? null,
    unit: 'index',
    description: 'UV Index from weather forecast'
  },

  // When satellite soil moisture is N/A, use weather soil data
  soil_moisture: {
    pillar: 'D',
    getValueFromExternal: (ext) => {
      const hourly = ext.weather?.hourly;
      if (!hourly) return null;
      const values = [
        hourly.soil_moisture_0_to_1cm?.[0],
        hourly.soil_moisture_1_to_3cm?.[0],
        hourly.soil_moisture_3_to_9cm?.[0]
      ].filter(v => v != null);
      return values.length ? values.reduce((a, b) => a + b) / values.length : null;
    },
    unit: 'm3/m3',
    description: 'Soil moisture from weather API'
  }
};
```

---

7. Database Architecture

Supabase (PostgreSQL)

```
-- Query History Table
CREATE TABLE query_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id TEXT NOT NULL,
    user_email TEXT,
    latitude DECIMAL(10, 6) NOT NULL,
    longitude DECIMAL(11, 6) NOT NULL,
    location_name TEXT,
    overall_score INTEGER,
    pillar_scores JSONB,
    query_mode TEXT DEFAULT 'comprehensive',
    pdf_generated BOOLEAN DEFAULT FALSE,
    pdf_url TEXT,
    pdf_downloaded BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Organizations Table
CREATE TABLE organizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    type TEXT, -- 'corporate', 'dfi', 'government', 'ngo'
    admin_email TEXT NOT NULL,
    subscription_tier TEXT DEFAULT 'basic',
```

```sql
    api_quota INTEGER DEFAULT 100,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Land Parcels Table
CREATE TABLE land_parcels (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID REFERENCES organizations(id),
    name TEXT NOT NULL,
    polygon_geojson JSONB NOT NULL,
    area_hectares DECIMAL(12, 4),
    latest_pps_score INTEGER,
    carbon_credits_tonnes DECIMAL(12, 2),
    esv_annual_usd DECIMAL(14, 2),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_query_history_user ON query_history(user_id);
CREATE INDEX idx_query_history_created ON query_history(created_at DESC);
CREATE INDEX idx_land_parcels_org ON land_parcels(organization_id);
```

Firebase Firestore (Sensor Data)

```typescript
// db/firestore.ts - Sensor data schema
interface SensorReading {
  device_id: string;
  timestamp: Timestamp;
  location: GeoPoint;
  readings: {
    temperature?: number;      // Celsius
    humidity?: number;         // Percentage
    soil_moisture?: number;    // m3/m3
    air_quality_index?: number;
    pm25?: number;            // ug/m3
    pm10?: number;            // ug/m3
    co2?: number;             // ppm
    noise_level?: number;     // dB
  };
  metadata: {
    battery_level?: number;
    signal_strength?: number;
    firmware_version?: string;
  };
}

// Real-time listener
const unsubscribe = onSnapshot(
  query(
    collection(db, 'sensor_readings'),
    where('device_id', '==', deviceId),
    orderBy('timestamp', 'desc'),
    limit(100)
  ),
```

```
  (snapshot) => {
    const readings = snapshot.docs.map(doc => doc.data() as SensorReading);
    updateDashboard(readings);
  }
);
```

---

8. API Specifications

REST API Endpoints

Method	Endpoint	Description	Auth
POST	/api/query	Point-based PPS query	Optional
POST	/api/query/polygon	Polygon-based query with ESV & Carbon	Optional
GET	/api/health	Health check	None
GET	/api/datasets	Available metrics list	None
GET	/api/imagery	Satellite imagery tiles	None
POST	/api/pdf	Generate PDF report	Required
GET	/api/history	User query history	Required
GET	/api/external/weather	Weather forecast	None
GET	/api/external/air-quality	Real-time AQI	None
GET	/api/external/soil	Soil moisture data	None

Request/Response Models

```
# app.py - Pydantic models
class PHIQueryRequest(BaseModel):
    latitude: float = Field(..., ge=-90, le=90)
    longitude: float = Field(..., ge=-180, le=180)
    mode: str = Field(default="comprehensive")
    temporal: str = Field(default="latest")
    buffer_radius: int = Field(default=500, ge=100, le=10000)
    pillars: Optional[List[str]] = None

    class Config:
        json_schema_extra = {
            "example": {
                "latitude": 28.6139,
                "longitude": 77.2090,
                "mode": "comprehensive",
                "buffer_radius": 500
            }
        }

class PHIQueryResponse(BaseModel):
    query: Dict[str, Any]           # Query metadata
    pillars: Dict[str, Any]         # A-E pillar data
```

```
    summary: PHISummary             # Aggregated scores
    time_series: Dict[str, Any]     # Temporal info
```

Polygon Query Response


```
{
  "query": {
    "type": "polygon",
    "points": [
      {"lat": 28.62, "lng": 77.20},
      {"lat": 28.62, "lng": 77.22},
      {"lat": 28.60, "lng": 77.22},
      {"lat": 28.60, "lng": 77.20}
    ],
    "centroid": {"latitude": 28.61, "longitude": 77.21}
  },
  "pillars": { /* ... pillar data ... */ },
  "summary": {
    "overall_score": 68,
    "overall_interpretation": "A",
    "ecosystem_type": "urban_green",
    "data_quality_score": 82.5,
    "geometry": {
      "area_ha": 125.4,
      "area_acres": 309.9
    },
    "carbon_credits": {
      "co2_equivalent_tonnes": 4520.5,
      "verified_co2_tonnes": 3730.8,
      "estimated_value": {
        "low_usd": 55962,
        "mid_usd": 93270,
        "high_usd": 186540
      },
      "methodology": "IPCC Tier 1"
    },
    "ecosystem_service_value": {
      "ecosystem_type": "urban_green",
      "base_esv_per_ha_usd": 3212,
      "phi_multiplier": 0.108,
      "adjusted_esv_per_ha_usd": 3559,
      "total_annual_esv_usd": 446298,
      "projections": {
        "10_year_usd": 4892156,
        "30_year_usd": 18052347
      }
    }
  }
}
```


---

9. Data Sources & Satellite Constellations

Primary Satellite Sources

Constellation	Agency	Sensors	Revisit	Data Type
MODIS	NASA	Terra/Aqua	Daily	Vegetation, Temperature, Atmosphere
Sentinel-2	ESA	MSI	5 days	Optical imagery (10-60m)
Landsat 8/9	USGS	OLI/TIRS	16 days	Optical + Thermal
GEDI	NASA	LiDAR	-	Canopy height, Biomass
SRTM	NASA/NGA	Radar	Static	Digital Elevation Model

Derived Products

Product	Source	Update	Use Case
Hansen Global Forest	U. Maryland	Annual	Tree cover, Forest loss
ESA WorldCover	ESA	Annual	Land cover classification
JRC Global Surface Water	JRC	Monthly	Water occurrence
Global Human Modification	CSP	Static	Anthropogenic impact
WorldPop	U. Southampton	Annual	Population density

Data Latency

Data Type	Latency	Notes
MODIS Composites	8-16 days	Pre-processed composites
Sentinel-2	2-5 days	Near real-time
Air Quality (OpenAQ)	Real-time	Ground station dependent
Weather Forecast	Real-time	Updated hourly
Annual Products	6-12 months	Hansen, WorldCover

---

10. Processing Pipeline

Query Execution Flow

```
1. REQUEST VALIDATION
   └─> Pydantic model validation
   └─> Coordinate bounds check
   └─> Mode validation (simple/comprehensive)

2. GEOMETRY CREATION
   └─> ee.Geometry.Point([lon, lat])
   └─> Buffer region (default 500m)
   └─> Or ee.Geometry.Polygon for land parcels

3. PARALLEL PILLAR QUERIES
   └─> ThreadPoolExecutor(max_workers=5)
   └─> Each pillar queries GEE independently
   └─> 15-35 second typical duration
```

```
4. METRIC EXTRACTION
   └> reduceRegion with mean reducer
   └> Scale factors applied
   └> Quality flags assigned

5. NORMALIZATION
   └> Select normalization function per metric
   └> Transform raw values to 0-100 scores

6. SCORING AGGREGATION
   └> Weighted pillar scores
   └> Ecosystem-adaptive weights
   └> Overall PPS score calculation

7. DATA QUALITY ASSESSMENT
   └> Criticality-weighted DQS
   └> Missing metrics identification
   └> Confidence recommendations

8. ECONOMIC VALUATION (Polygon only)
   └> ESV calculation with PHI multiplier
   └> Carbon credit estimation
   └> 10/30 year projections

9. RESPONSE ASSEMBLY
   └> JSON serialization
   └> Metadata attachment
   └> Query history logging
```

Performance Benchmarks

Operation	Typical Duration	Notes
GEE Authentication	2-3s	First request only
Single Pillar Query	3-8s	Depends on metrics
Full Parallel Query	12-25s	All 5 pillars
Simple Mode Query	8-15s	Reduced metrics
Polygon Area Calc	1-2s	GEE geometry operations
ESV/Carbon Calc	<100ms	Local computation
PDF Generation	5-10s	Server-side rendering

---

11. Security & Authentication

Authentication Methods

1. **Firebase Authentication** (Frontend)
   - Email/Password
   - OAuth providers (Google, GitHub)
   - JWT tokens for API calls

2. **API Key Authentication** (Backend)
    - Organization-level API keys
    - Rate limiting per key
    - Usage tracking
3. **GEE Service Account** (Backend)
    - Service account credentials
    - Project-level quotas
    - Compute unit monitoring

Security Headers

```
# CORS Configuration
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",
        "http://localhost:5173",
        "https://app.erthaloka.com"
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Input Validation

```
# Coordinate validation
latitude: float = Field(..., ge=-90, le=90)
longitude: float = Field(..., ge=-180, le=180)
buffer_radius: int = Field(default=500, ge=100, le=10000)

# Pillar validation
valid_pillars = {"A", "B", "C", "D", "E"}
if request.pillars:
    invalid = set(request.pillars) - valid_pillars
    if invalid:
        raise HTTPException(400, f"Invalid pillars: {invalid}")
```

---

12. Deployment Architecture

Production Deployment

```
                    ┌──────────────────┐
                    │   CloudFlare     │
                    │   CDN / WAF      │
                    └──────────┬───────┘
                               │
            ┌──────────────────┴──────────────────┐
            │                                     │
   ┌────────▼─────────┐                ┌─────────▼────────┐
   │  Vercel / Netlify │                │  Cloud Run /     │
```

```
    │  Frontend (React) │             │  Railway        │
    │                   │             │  FastAPI Backend│
    └─────────┬─────────┘             └────────┬────────┘
              │                               │
              │                    ┌──────────┴──────────┐
              │                    │                     │
              │          ┌─────────▼─────────┐   ┌───────▼─────────┐
              │          │  Google Earth     │   │  External APIs  │
              │          │  Engine           │   │  Open-Meteo     │
              │          │                   │   │  OpenAQ         │
              │          └───────────────────┘   └─────────────────┘
              │
              │
    ┌─────────▼─────────────────────────────────┐
    │                  Supabase                 │
    │  PostgreSQL + Auth + Realtime             │
    └───────────────────────────────────────────┘
              │
    ┌─────────▼─────────────────────────────────┐
    │              Firebase                     │
    │  Firestore (Sensor Data) + Auth           │
    └───────────────────────────────────────────┘
```

Environment Variables

```
# Frontend (.env)
VITE_PHI_API_URL=https://api.erthaloka.com
VITE_FIREBASE_API_KEY=xxx
VITE_FIREBASE_PROJECT_ID=erthaloka
VITE_SUPABASE_URL=https://xxx.supabase.co
VITE_SUPABASE_ANON_KEY=xxx

# Backend (.env)
GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account.json
GEE_PROJECT_ID=erthaloka-gee
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_SERVICE_KEY=xxx
OPENAQ_API_KEY=xxx
```

Monitoring & Observability

Tool	Purpose
Cloud Logging	Application logs
Cloud Monitoring	Performance metrics
Sentry	Error tracking
Uptime Robot	Availability monitoring

Summary

The ErthaLoka PPS is built on a modern, scalable architecture that combines:

1. **React + TypeScript frontend** with real-time data visualization
2. **FastAPI backend** orchestrating parallel satellite data queries

3. **Google Earth Engine** providing access to 50+ petabytes of geospatial data
4. **External API integration** for real-time weather and air quality
5. **PostgreSQL + Firestore** for persistent and real-time data storage
6. **Production-grade security** with Firebase Auth and API key management

The system processes 24 environmental indicators across 5 pillars, applying scientifically-validated normalization algorithms to produce a single PPS score with AAA-CCC grading, comparable to financial credit ratings.

Document Version: 2.0 Last Updated: December 2024 Part of PPS Technical Deep Dive Series